Defiende el servidor

MLOps · Visualización 2 · 2026-II · Grupo B — Luna Lugo, Alejandro Bernal · Universidad Externado de Colombia
Código: `github.com/alejobernalg/defiende-el-servidor` · Modelos: `huggingface.co/AlejandroLiberato/defiende-el-servidor-unsw-nb15` · DVC: `huggingface.co/datasets/AlejandroLiberato/defiende-el-servidor-dvc-storage`

> Cadena de trazabilidad: **commit Git → referencia DVC** (md5 de `data/prepared.dvc`) → **run ID de MLflow → revisión SHA de Hugging Face**. Cada pieza se vincula por identificadores y hashes, no por nombres de archivo. Todas las cifras de este informe provienen de los archivos en `evidencias/`.

1. Datos, auditoría y limpieza

Se usaron las particiones publicadas de UNSW-NB15. La copia descargada (mirror en GitHub) tenía **los nombres de archivo intercambiados**: el archivo llamado *training* tenía 82.332 filas y la huella del test del anexo. Cada archivo se asignó por SHA-256 y número de filas, no por su nombre (`data/fuentes.json`). Ambas huellas coinciden con el anexo.

Archivo	Filas	Cols.	Normal / Ataque	Faltantes	Duplicados por entradas*	Grupos contradictorios (filas)
training-set (`bec7dd5e…48fa`)	175.341	45	56.000 / 119.341	0	74.301	229 (940)
testing-set (`734fe664…a559`)	82.332	45	37.000 / 45.332	0	28.386	6 (14)

* Filas repetidas más allá de la primera, comparando solo las 42 variables predictoras (sin `id`, `attack_cat` ni `label`).

- **Variables:** 42 predictoras (3 categóricas: `proto`, `service`, `state`; 39 numéricas). Excluidas: `id` (identificador, solo para trazabilidad), `attack_cat` (revela la respuesta) y `label` (objetivo; ataque = 1 es la clase positiva).
- **Política de limpieza:** no se elimina ninguna fila. Los duplicados reflejan la frecuencia real del tráfico. Los grupos contradictorios se conservan porque quitarlos exige mirar la etiqueta y el test también los contiene; son error irreducible y se reportan.
- **Particiones:** solo a partir del archivo de entrenamiento. Cada fila recibe `group_id` = hash de sus entradas y `GroupShuffleSplit` (20 %, semilla 42) asigna grupos completos: **train 140.145 filas** (80.832 grupos) y **validación 35.196 filas** (20.208 grupos), con **0 grupos compartidos**. La asignación fila a fila está en `data/prepared/splits.csv` (`row_id = train:<i>`).
- **Sin fuga:** imputación, one-hot, log1p y escalado viven dentro de un `sklearn.Pipeline` ajustado solo con train; el pipeline completo se guarda como un único artefacto.
- **Solapamiento con test:** 8.541 filas de test (10,4 %) tienen entradas idénticas a filas de train/val (6.937 de train y 1.604 de val). Las métricas de test incluyen esas filas y se interpretan con esa limitación.
- **Dos versiones DVC:** `data/raw.dvc` (archivos recibidos, md5 `fe93051a…`, commit `119c57a`) y `data/prepared.dvc` (train/val/splits/manifest, md5 `f6594eb8…`, commit `8a7a81f`). El remote es un dataset de Hugging Face leído por HTTP: `dvc pull` funciona sin credenciales ni caché previa.

2. Cuatro experimentos, dos familias

Todos usan la misma partición, semilla 42 y umbral 0,5 para compararse en igualdad de condiciones. Cada run registra en MLflow la hipótesis, el cambio, los parámetros, la semilla, el commit (`ecbb489`), los md5 DVC, los SHA-256 de train/val, las métricas, la matriz de confusión, la curva de umbrales y el modelo completo. `train.py` se niega a correr con cambios sin commit. Las métricas usan ataque como clase positiva; si un denominador es cero, la métrica vale 0,0 y se lista en `zero_denominators` (no ocurrió).

ID · run	Familia / cambio · hipótesis	Recall	Prec.	FPR	Macro-F1	FP / FN
E1 `ae051ab6…`	Reg. logística, one-hot + escalado (referencia). Un modelo lineal ya separa la mayoría de ataques.	0,987	0,925	0,174	0,923	1.937 / 314
E2 `04da9b28…`	Igual + `log1p` en numéricas. Las colas pesadas perjudican al modelo lineal.	0,974	0,940	0,134	0,929	1.494 / 631

E3 `7bc3f16d…`	Árbol de decisión (prof. 12, hoja ≥ 20). Hay umbrales e interacciones no lineales.	0,971	0,952	0,106	0,938	1.182 / 687
E4 `ef3b2eaf…`	Random forest, 300 árboles. Promediar árboles reduce la varianza de E3.	0,979	0,956	0,097	0,947	1.079 / 511

Interpretación. E1 detecta casi todos los ataques, pero marca como ataque el 17 % del tráfico normal: en un SOC eso significa miles de alertas falsas. `log1p` confirma su hipótesis en parte: baja la FPR 4 puntos a costa de recall. Pasar a árboles es el cambio más útil (FPR 0,106 y luego 0,097), lo que indica que las relaciones no son lineales (p. ej. `sttl` × `state`). E4 tiene además el mayor ROC-AUC (0,993), es decir, puntuaciones que ordenan mejor y permiten mover el umbral con poco costo.

3. v1 y v2: por qué existen y qué cambia

	v1	**v2**
Solución	E1, umbral 0,50 (primera solución funcional)	E4, umbral 0,64 (revisión basada en los experimentos)
Código	tag `code-v1` = `ecbb4899…`	tag `code-v2` = `43878a7f…`; selección congelada en `f7d4ffa3…`
MLflow run	`ae051ab637994bb3bd29cf521f544087`	`ef3b2eaf90874ea5ae5b4b867327f00d`
HF revisión	`8033bee6b22195aa79e007085d680b104256a769`	`6282a4d6e05d473f37d0243f3bfa18b10116586e`
SHA-256 modelo	`2f8deb7a…45c0`	`03d39889…8a81`
Validación	recall 0,987 · prec. 0,925 · FPR 0,174 · macro-F1 0,923 · FP 1.937 · FN 314	recall 0,951 · prec. 0,977 · FPR 0,049 · macro-F1 0,944 · FP 548 · FN 1.176

Qué mejora: las alertas falsas en validación pasan de 1.937 a 548 (–72 %) y la precision sube a 0,977. **Qué empeora:** se pierden 1.176 ataques en lugar de 314. Se acepta este costo porque el equipo de seguridad no puede revisar un 17 % de falsas alarmas, y la regla garantiza recall ≥ 0,95. Cada revisión de Hugging Face contiene el pipeline completo, `config.json` (umbral, variables, hashes y referencias Git/DVC/MLflow), `predict.py`, `requirements.txt` y la model card.

4. Elección del umbral (solo validación)

La regla se escribió en `configs/seleccion.yaml` y se hizo commit **antes** de ejecutar los experimentos:

- **Modelo:** el de mayor macro-F1 en validación con umbral 0,5 (desempate por menor FPR).
- **Umbral:** el que **minimiza la FPR manteniendo recall de ataque ≥ 0,95**, en una rejilla de 0,01 a 0,99 (`y_pred = 1 si score_attack ≥ umbral`).

Con E4, pasar de 0,50 a 0,64 baja la FPR de 0,097 a 0,049 con un recall de 0,951. El macro-F1 baja levemente (0,947 → 0,944): el criterio prioriza menos alertas falsas sobre macro-F1, y la decisión se documenta. `evidencias/seleccion.json` se guardó con un commit fechado (2026-10-29) antes de cualquier uso del test, y el test no intervino en ninguna decisión.

5. Evaluación final en test

`evaluate_test.py` descarga v2 de Hugging Face por su SHA, comprueba que el hash del modelo coincide con la selección congelada y evalúa **una sola vez** las 82.332 filas de test, sin eliminar ninguna. Las predicciones por fila (`row_id = test:<i>`, `y_true`, `score_attack`, `y_pred`) están en `predicciones-test.csv`; `recompute_test_metrics.py` recalcula las métricas solo con la librería estándar y coinciden exactamente.

Conjunto (umbral 0,64)	**Recall**	**Precision**	**FPR**	**Macro-F1**	**TN / FP / FN / TP**
Validación	0,951	0,977	0,049	0,944	10.575 / 548 / 1.176 / 22.897
Test	0,966	0,873	**0,173**	0,901	30.604 / 6.396 / 1.558 / 43.774

El resultado empeora y se conserva. La FPR se triplica respecto a validación. El análisis descriptivo posterior (`analisis-errores.json`, no usado para ajustar nada) muestra dos causas:

- El test tiene más tráfico normal (45 % frente a 32 %).
- El tráfico normal con `service='-'` recibe una FPR de 0,20 en test frente a 0,06 en validación; también `http` (0,17) y `ftp` (0,28).

Es un cambio de distribución entre las particiones publicadas que la validación interna, tomada del mismo archivo de entrenamiento, no podía anticipar. Por tipo de ataque, el recall es ≥ 0,98 salvo en *Fuzzers* (0,76). Al añadir el resultado a la model card se creó una nueva revisión de HF (`bfdc6b94…`) que solo cambia `README.md`; se verificó que `model.joblib` conserva el SHA-256 `03d39889…`.

6. Prueba de recuperación

`scripts/recuperacion.sh` se ejecutó en una carpeta y un entorno nuevos: clona GitHub en el commit de entrega, crea un venv nuevo, instala `requirements.txt` y ejecuta `dvc pull` desde Hugging Face con la caché vacía. Luego verifica las huellas de los datos raw y preparados, descarga v1 y v2 por su SHA completo en una caché de HF vacía y predice 25 filas fijas de validación (10 normales y 15 ataques reales). Las salidas recuperadas se comparan con las guardadas antes de publicar: `y_pred` idéntico y $|\Delta \text{ score}| \leq$ 1e-9. Ese margen existe porque el random forest suma en paralelo y la diferencia observada es del orden de 1e-16. El log, las entradas, las salidas, los hashes y las comparaciones están en `evidencias/recuperacion/`.

7. Limitaciones

- UNSW-NB15 es un banco de pruebas sintético de 2015: no demuestra eficacia en una red actual.
- Hay un cambio de distribución entre las particiones publicadas (sección 5); un despliegue real requeriría recalibrar el umbral con tráfico propio y monitorear la FPR.
- El umbral se eligió en la misma validación donde se reportan las métricas de v2, así que estas son algo optimistas; el test lo confirma.
- El 10,4 % del test repite entradas vistas en train/val, y hay grupos con etiquetas contradictorias.
- Los modelos son pickles de scikit-learn: solo deben cargarse con las versiones fijadas y desde fuentes de confianza.
- El seguimiento de MLflow es local (SQLite); la evidencia consultable es la exportación en `evidencias/mlflow/`.

Referencias

Moustafa, N., & Slay, J. (2015). UNSW-NB15: a comprehensive data set for network intrusion detection systems. *MilCIS*, IEEE. · Moustafa, N., & Slay, J. (2016). The evaluation of Network Anomaly Detection Systems: Statistical analysis of the UNSW-NB15 data set and the comparison with the KDD99 data set. *Information Security Journal*, 25(1–3). · Documentación de scikit-learn (Pipeline, GroupShuffleSplit, «Common pitfalls»), DVC, MLflow Tracking y Hugging Face Hub (descarga por revisión, model cards). · Asistencia de IA: Claude Code (Anthropic), declarada en `entrega.md`.